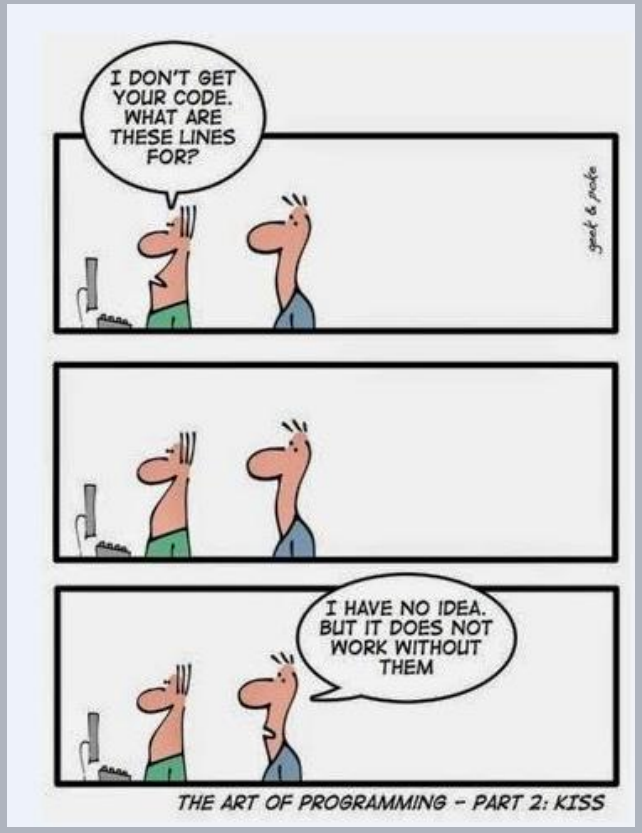
Advanced Software Design Techniques

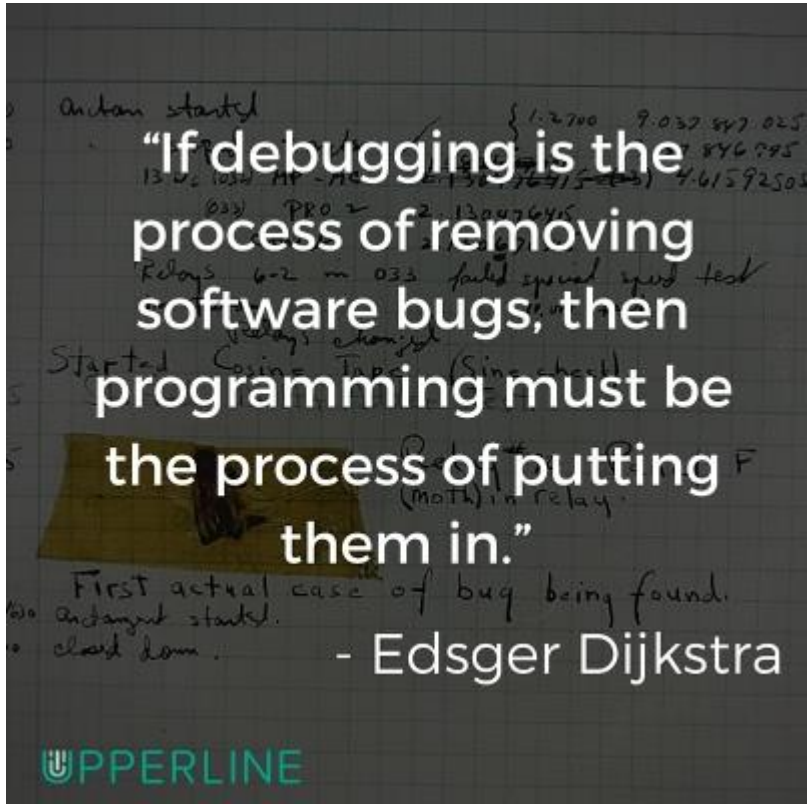
Error Handling

Prof. Dr.-Ing. Peter Fromm



Content

- Motivation for error handling
- Where are problems born?
- Fault Detection
- Error Handling Techniques
  - Return Code
  - C++ try/catch
  - Callback
  - Central Handler
  - Trap Handler
- Debugging



Close to Xmas – why are we doing this?

Software Bug

A **software bug** is an error, flaw, mistake, failure, fault or "undocumented feature" in a computer program that prevents it from behaving as intended (e.g. , producing an incorrect or unexpected result).

Most bugs arise from mistakes and errors made by people in either a program's source code or its design, and a few are caused by compilers producing incorrect code. A program that contains a large number of bugs, and/or bugs that seriously interfere with its functionality, is said to be buggy. Reports detailing bugs in a program are commonly known as bug reports, fault reports, problem reports, trouble reports, change requests, and so forth.

Deadly Accident Stint – 23.09.2018

4 kids in the age between 4 and 8 die, the driver and another kid survive heavily wounded.

Root Cause Analysis

2 Die Untersuchung

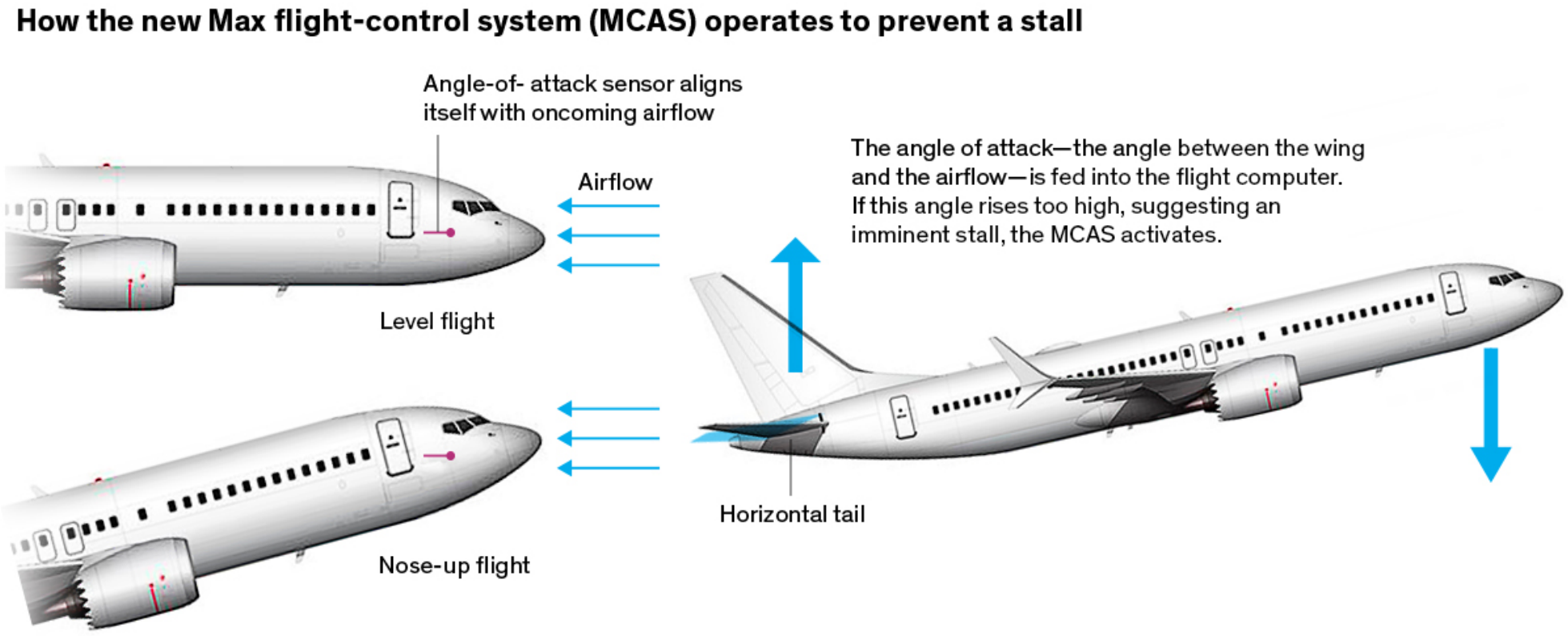
Nach dem Unglück ordnete die zuständige Ministerin Cora van Nieuwenhuizen eine Untersuchung an. Die Ergebnisse liegen seit Anfang Oktober vor. Demnach

- kann es beim elektrisch betriebenen Stint zu einem Kabeldefekt und **Kurzschluss** kommen. Das Fahrzeug beschleunigt dann auf **17 Stundenkilometer** und ist kaum mehr zu beherrschen. Auch die eingebaute Handbremse kann das Gefährt bei **diesem Tempo nicht mehr stoppen**.
- Der Fahrer kann den Stint dann nur noch stoppen, indem er den Schlüssel zieht. "Aber das ist in einer solchen **Paniksituation** nicht wahrscheinlich", schreiben die technischen Prüfer.
- Der Hersteller, ein niederländischen Unternehmen, kannte nach Angaben der Untersuchung die Gefahren, handelte aber nicht. Zudem soll er ohne Genehmigung seit 2014 stärkere Motoren in den Stint eingebaut haben.

- A cable broke, causing a short circuit
- This increased the speed to 17km/h
- Even when using the handbrake, the vehicle could not be stopped.
- The problem was known since 2014!

https://www.watson.de/leben/niederlande/257601211-stint-darum-verbieten-die-niederlanden-den-elektrischen-lastenroller, Access 10.10.2018

Boing MCAS Disaster - 2019



Rout Cause Analysis

So Boeing produced a dynamically unstable airframe, the 737 Max. That is big strike No. 1. Boeing then tried to mask the 737's dynamic instability with a software system. Big strike No. 2. Finally, the software relied on systems known for their propensity to fail (angle-of-attack indicators) and did not appear to include even rudimentary provisions to cross-check the outputs of the angle-of-attack sensor against other sensors, or even the other angle-of-attack sensor. Big strike No. 3.

None of the above should have passed muster. None of the above should have passed the "OK" pencil of the most junior engineering staff, much less a DER.

That's not a big strike. That's a political, social, economic, and technical sin.

https://spectrum.ieee.org/aerospace/aviation/how-the-boeing-737-max-disaster-looks-to-a-software-developer

Motivation Error Handling

Many system failures are caused by wrong or incomplete error handling!

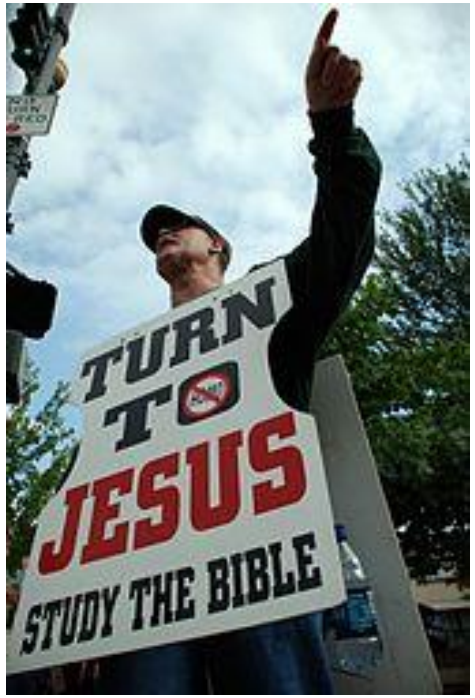
Incomplete specifications

System complexity

Typically high number of potential malfunctions

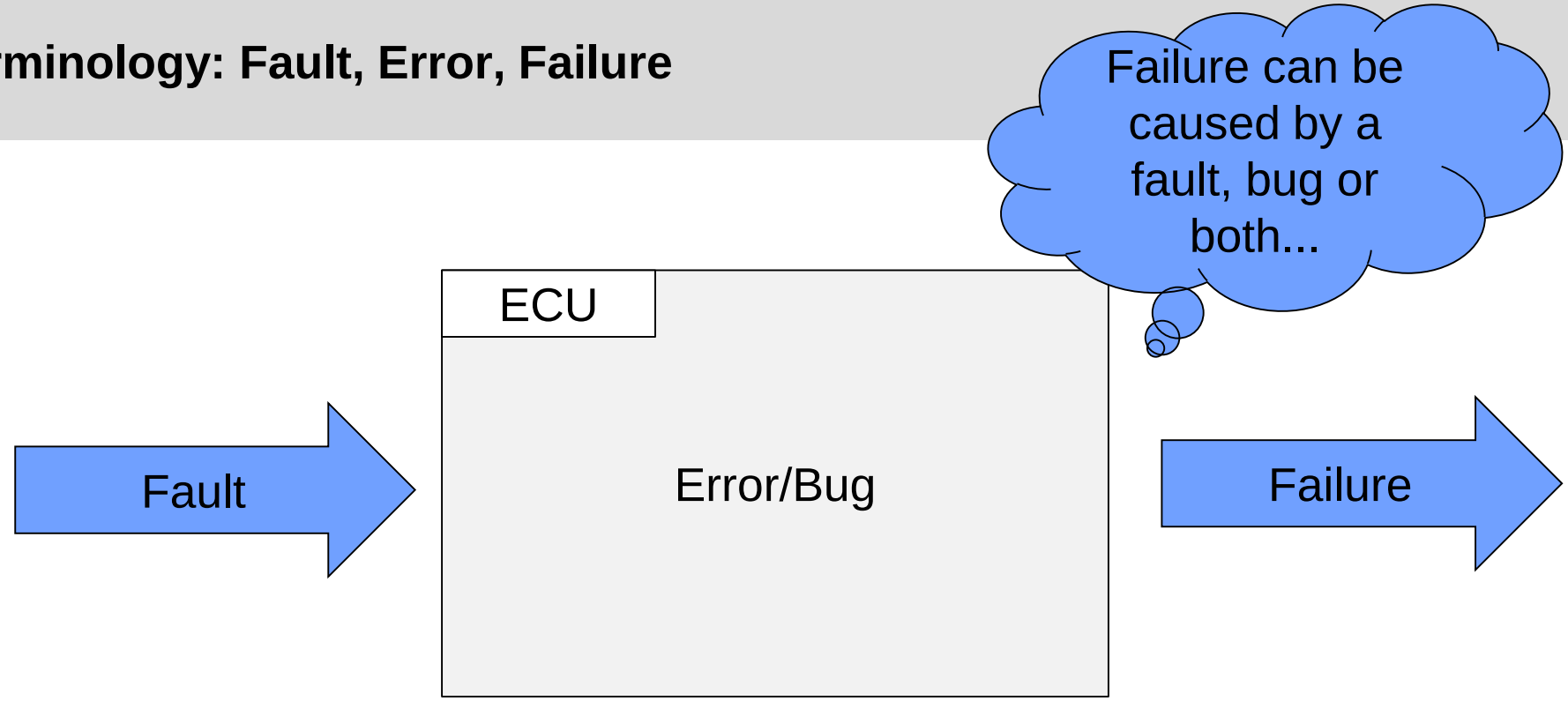
Unclear how to handle errors – so let's ignore it

Costs!



Yesterday at my local train station

Some Terminology: Fault, Error, Failure

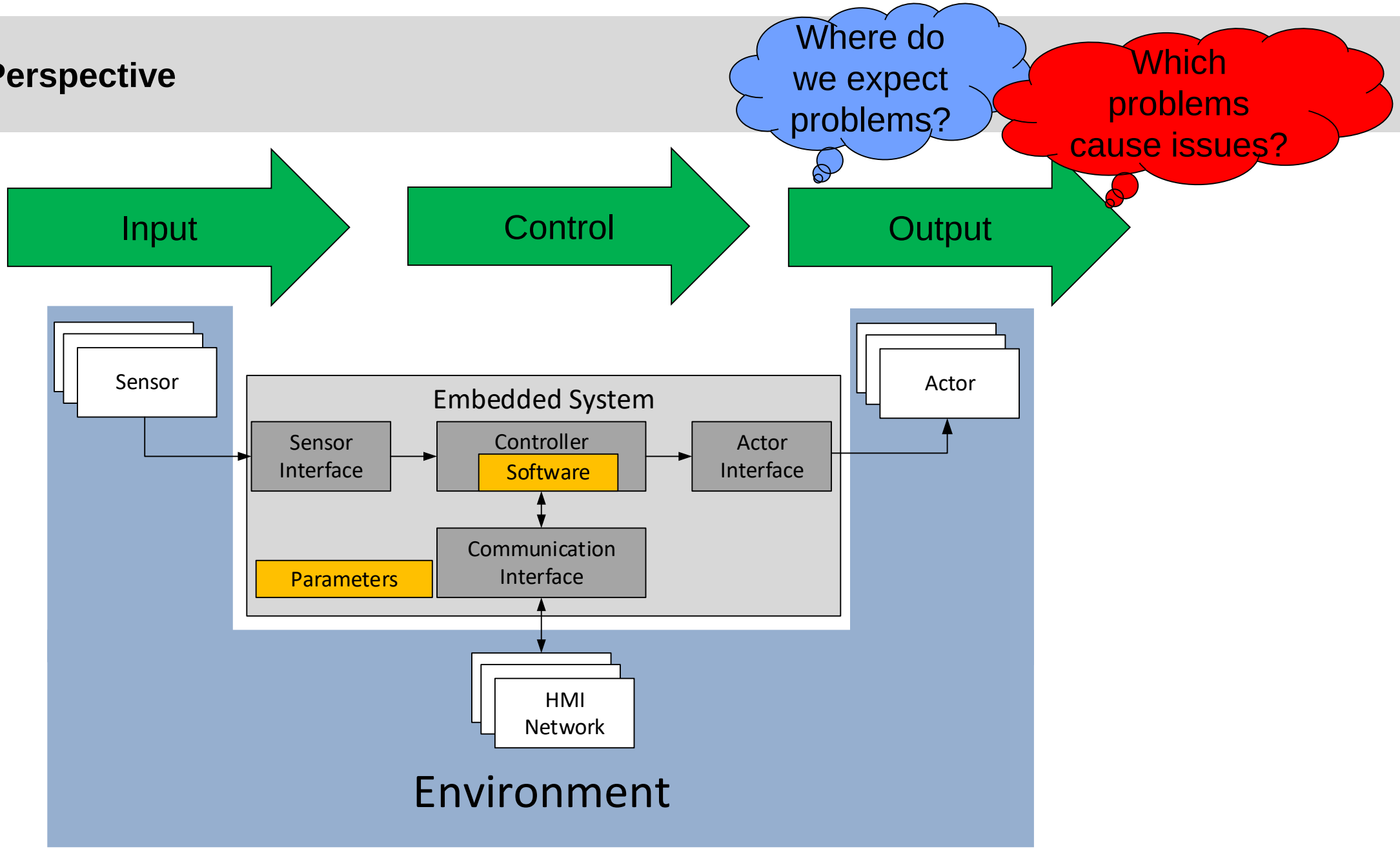


e.g.
- Wrong sensor Value
- Wrong user Input
- Transmission error
- ...

e.g.
- Logical error
- Data overflow
- Race condition
- ...

e.g.
- Wrong data
- Wrong actor motion
- Wrong protocol
- Wrong HMI output...

System Perspective



Sensor

Potential Faults:

- Incorrect monitoring / measurement of the environment
- Corrupted transmission to the controller
- Wrong perception through the port / driver
- Too old data

System requirements:

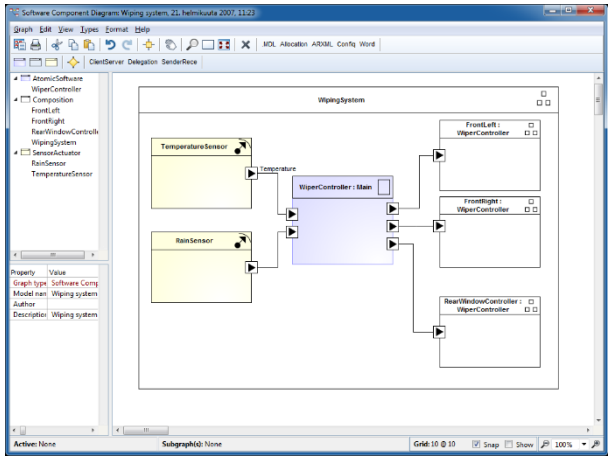
- We must detect wrong sensor values
- We must identify corrupt transmission packages
- We must verify the correct operation of the receiver port
- We must know how old the data is

And then we have to decide how to react on this wrong data!

MCU Hardware / Software

Potential Faults:

- Wrong logic, wrong data types, wrong calculations,….
- Race conditions, real time violations, program hanging
- Memory leaks, wild pointers…
- Hardware failure



System Requirements:

- Systematic software bugs must be avoided
- Stochastic software bugs (memory overflow, realtime violations, memory corruption) must be detected
- Freedom from interference (memory, CPU, peripherals) must be ensured between code of different SIL levels
- Hardware bugs (memory, CPU,…) must be detected

Can we develop a complex software system without bugs?

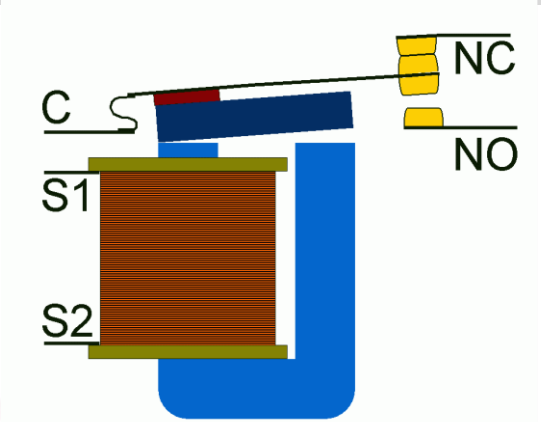
Actor

Potential Faults:

- Incorrect drive / port
- Data corruption during transmission to the actor
- Hardware failure of the actor

System Requirements:

- The behaviour of the actor must be diagnosable
- We need to ensure a safe state of the actor



How can we control the actor if it is obviously out of control? Fail-Safe versus Fail-Operational?

HMI / Network

Potential Faults:

- Incorrect usage of driver / port
- Incorrect protocol
- Transmission errors / network failure
- Incompatibility or wrong state of receiver
- Incorrect windows management, wrong displacements
- Frozen display

System requirements:

- Correct transmission must be diagnosable
- Correct reception and understanding of data must be ensured

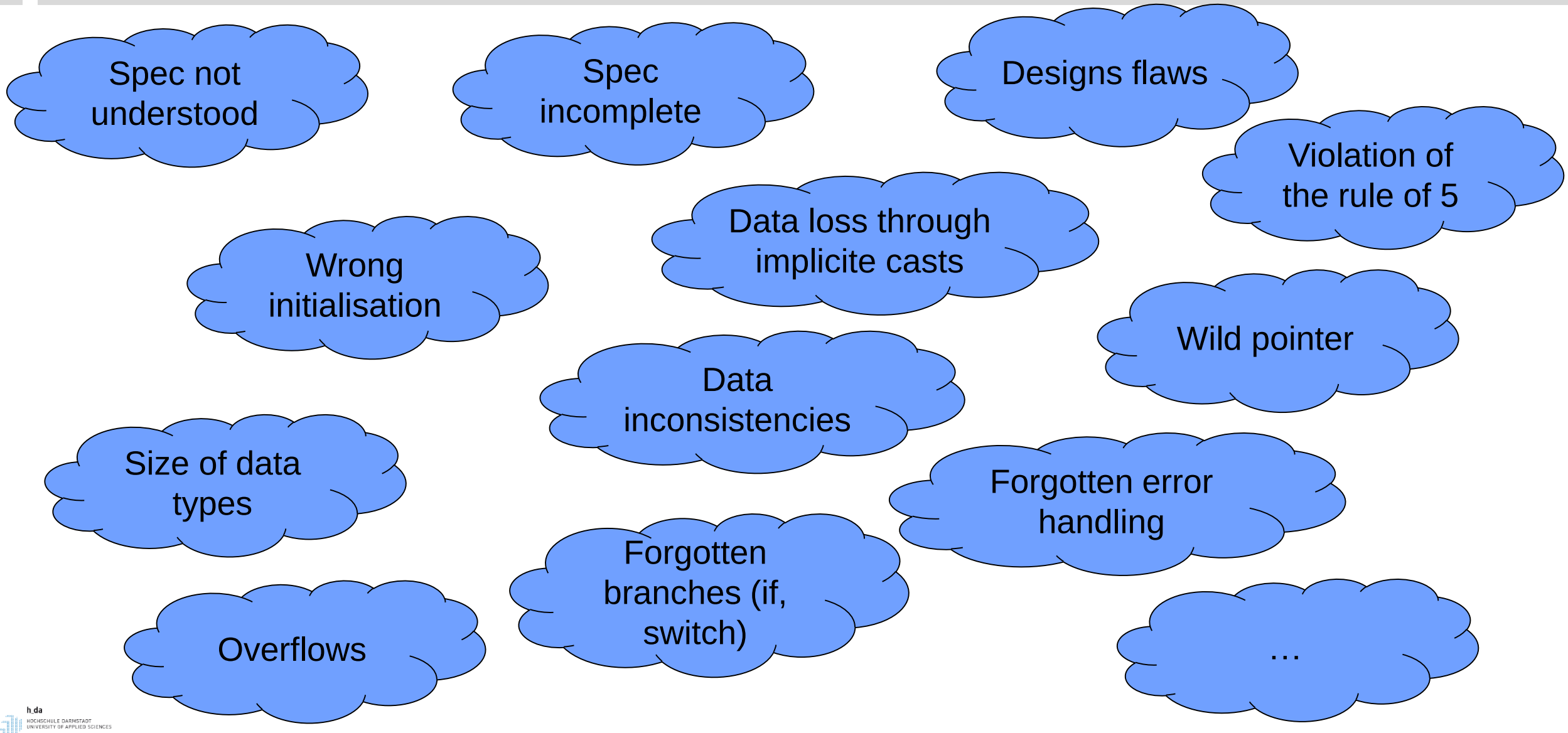
Supervision of displays / network is hard to achieve!

Top level requirements

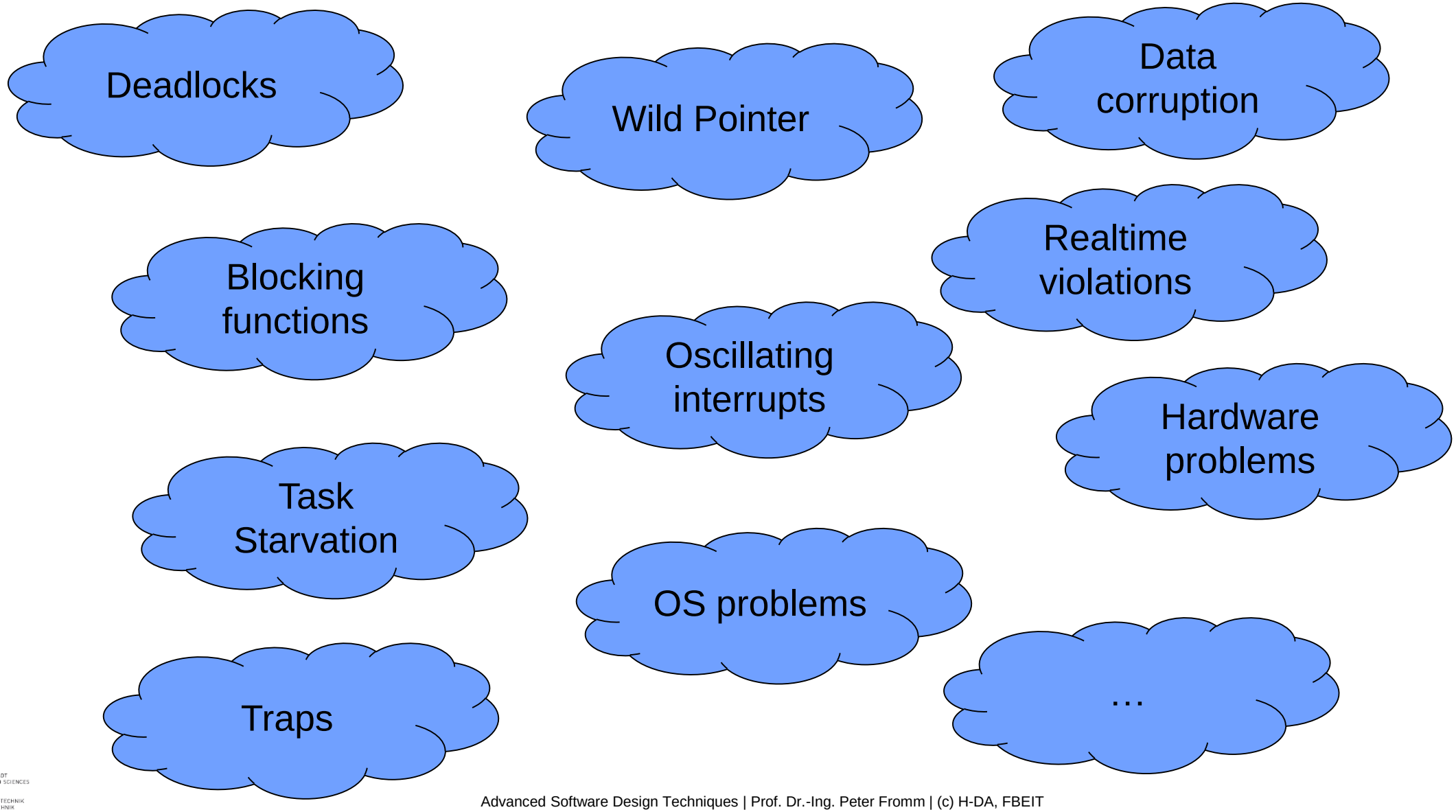
Reliable detection and handling of stochastic faults and bugs.

Minimization of systematic errors.

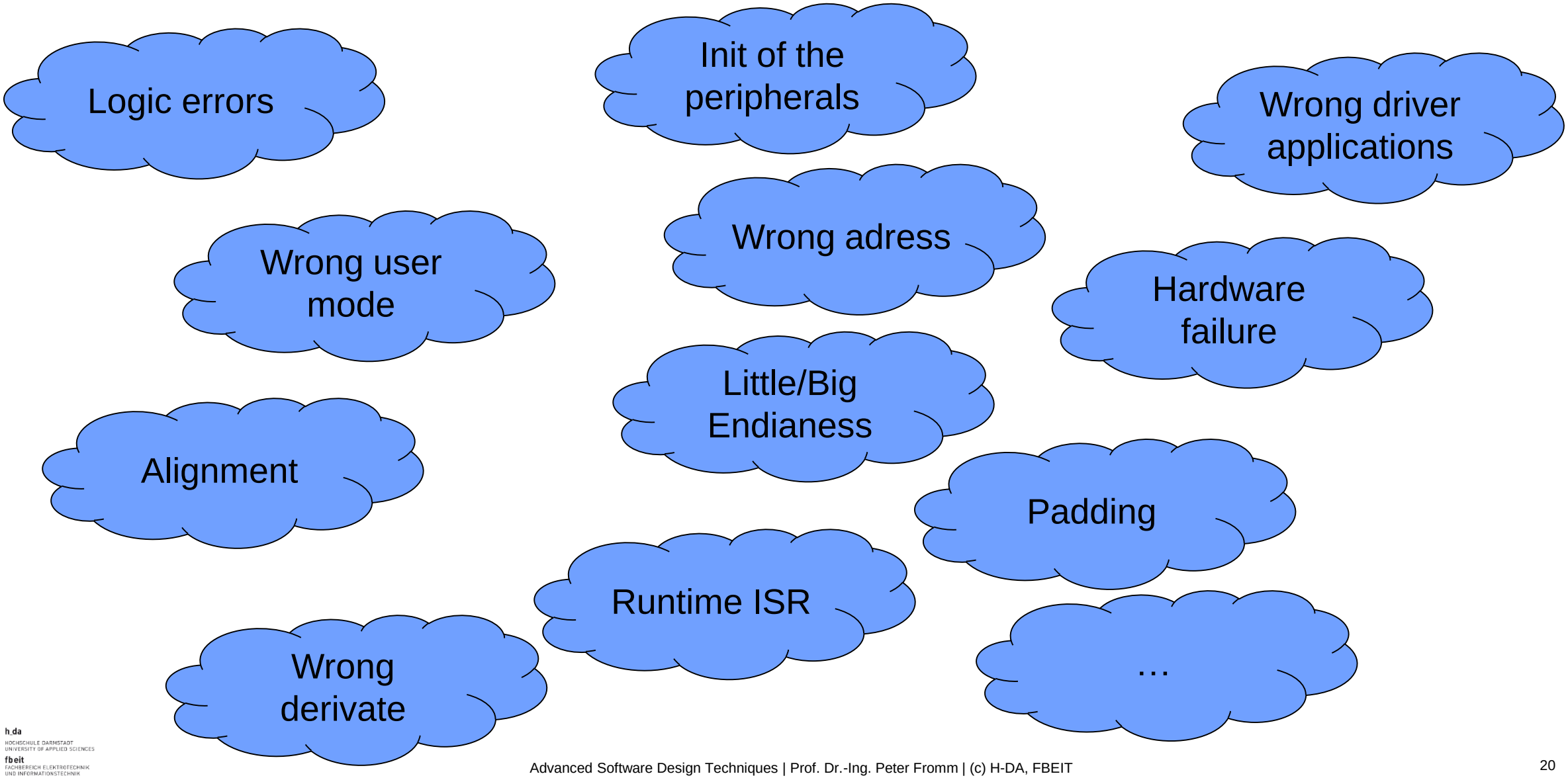
Brain Storming – Causes for logical (systematic) errors



Brain Storming – Causes for stochastic errors

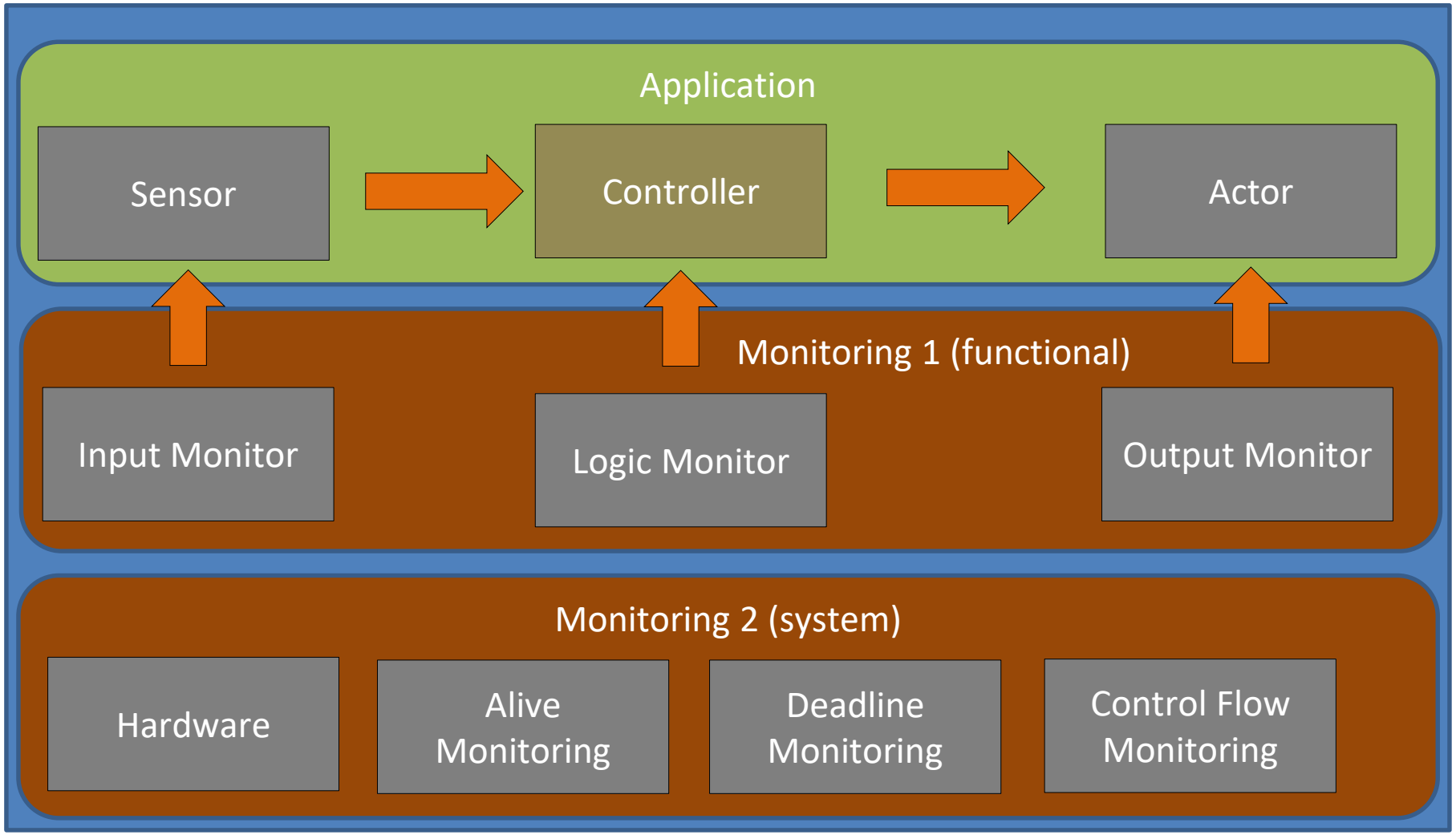


Brain Storming – Failure of the hardware interface / port



It’s your responsibility to deal with this challenge!

Typical error handling architecture



Identifying faults on application level

Identifying faults is highly application specific, but some patterns can be generally applied

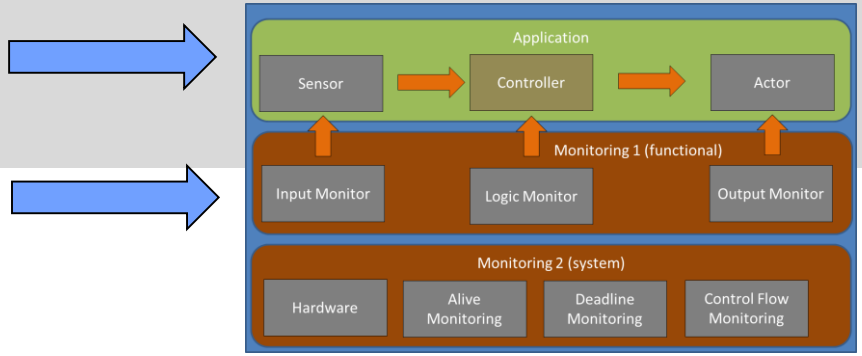
Sensor faults
- Use redundant sensors (1oo2, 2oo3)
- Use different technologies

Data / communication faults
- Range check of data
- CRC code
- Age of data (timestamp of update)

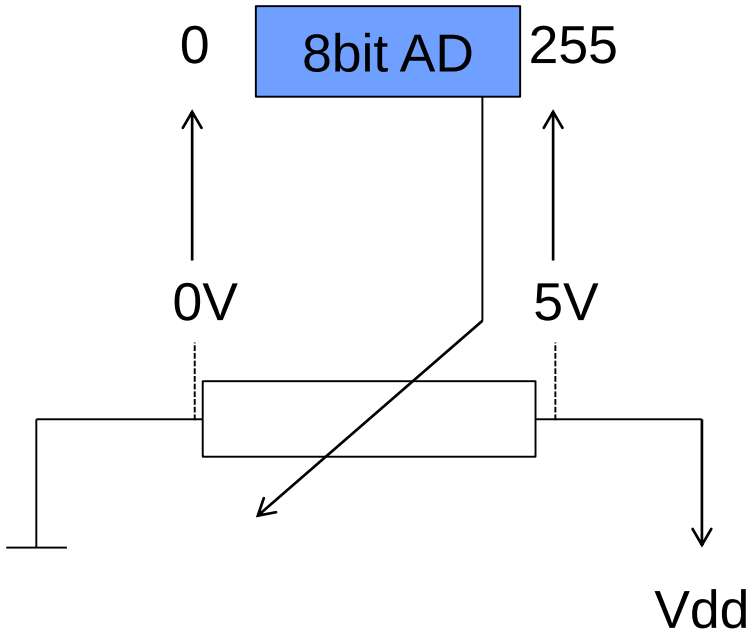
Logical faults
- Check pointers for validity
- Check results for plausibility
- Check return values / error codes of functions

Once a fault has been identified, we must handle it

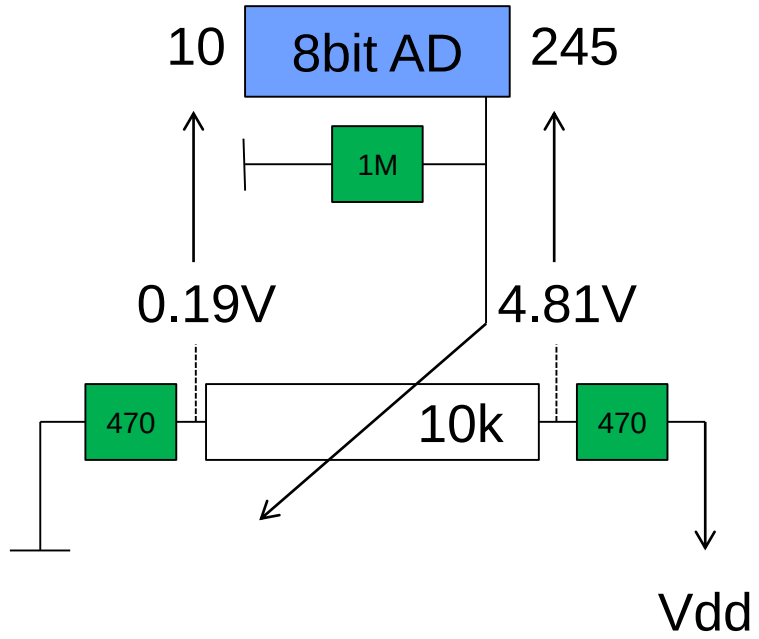
Identifying faults on application level
Creating a “valid range”



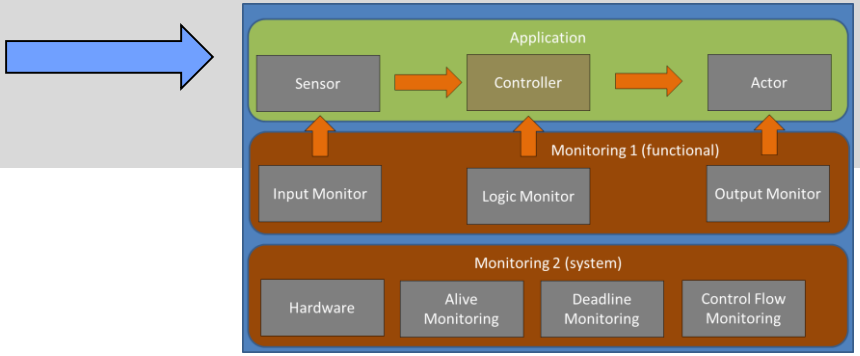
Range: 0..255
Valid Range: 0..255



Range: 0..255
Valid Range: 11..244
0..10 and 245..255 show errors

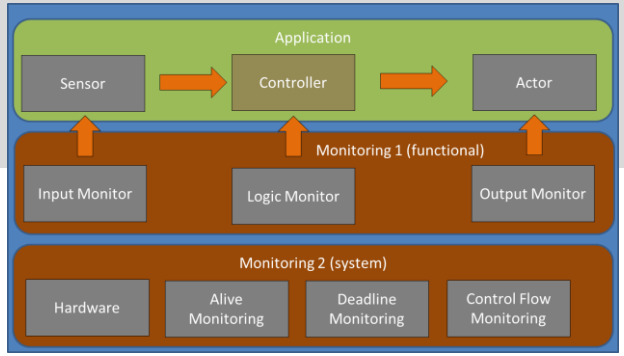


Monitor Functions
Traditional Implementation using hierarchical error handling




```
bool control(int targetvalue, int* controlvalue)
{
// Check for valid target value
if (targetvalue < TARGETMIN || targetvalue > TARGETMAX)
{
        //error, control value will not be changed
        return false;
}

//Simple P-controller
*controlvalue += (targetvalue - getCurrentValue()) * K;

return true;
}

if (true==control(targetvalue, &controlvalue))
{
        //Engine will only be set in case of no error
        setEngine(controlvalue)
}
```

Monitor Function
Separation of range check and controller function

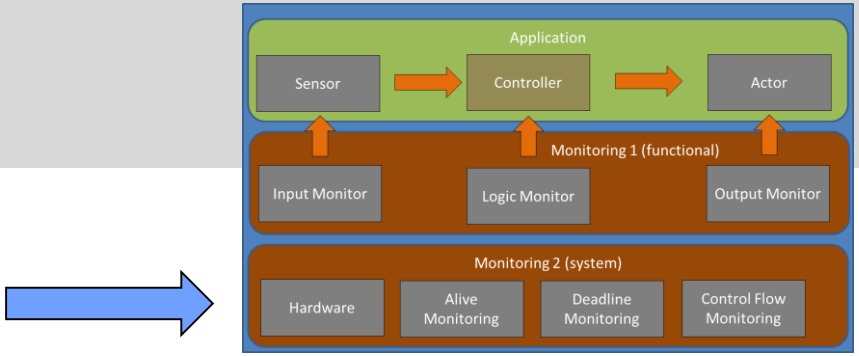



```
bool checkTarget(int targetvalue)
{
if (targetvalue < TARGETMIN || targetvalue > TARGETMAX) return false;
return true;
}

void control(int targetvalue, int* controlvalue)
{
*controlvalue += (targetvalue – getCurrentValue()) * K;
}

if (true == checkTarget(targetValue)
{
        control(targetValue, &controlvalue);
        setEngine(controlvalue);
}
```

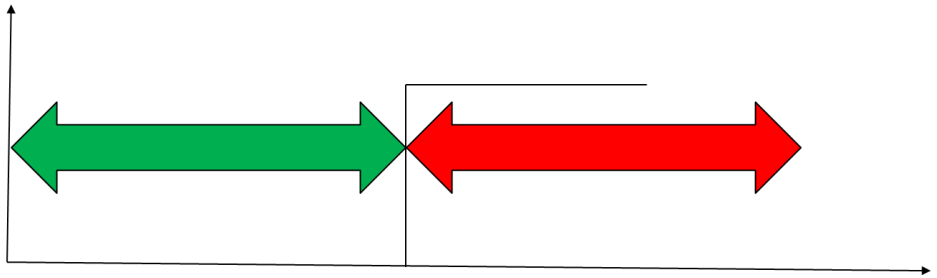
When using central data pool as architectural concept, like the AUTOSAR RTE, the monitors can check the validity of the signals independently.

Alive, Deadline, Control Flow Monitoring

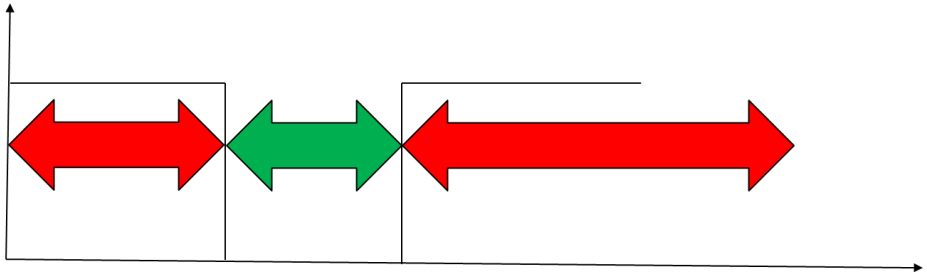
```
uint8_t watchdogpattern = 0;

TASK(task_cycle1)
{
          watchdogpattern |= 0x01;       //Bit 1

//Some more code
}

TASK(task_cycle2)
{
          watchdogpattern |= 0x02;       //Bit 2

//Some more code
}


//Low priority
TASK(task_background)
{
while(1) {
          if (watchdogpattern == 3) {
                    TriggerWatchdog();
                    watchdogpattern = 0;
        }
}
}
```



Timeout Watchdog



Window Watchdog



Error handling

Exception handling is the process of responding to the occurrence, during computation, of *exceptions* – anomalous or exceptional conditions requiring special processing – often disrupting the normal flow of program execution. It is provided by specialized programming language constructs, computer hardware mechanisms like interrupts or operating system IPC facilities like signals.

In general, an exception breaks the normal flow of execution and executes a pre-registered *exception handler*. The details of how this is done depends on whether it is a hardware or software exception and how the software exception is implemented. Some exceptions, especially hardware ones, may be handled so gracefully that execution can resume where it was interrupted.

Alternative approaches to exception handling in software are error checking, which maintains normal program flow with later explicit checks for contingencies reported using special return values or some auxiliary global variable such as C's errno or floating point status flags; or input validation to preemptively filter exceptional cases.

(Wikipedia)

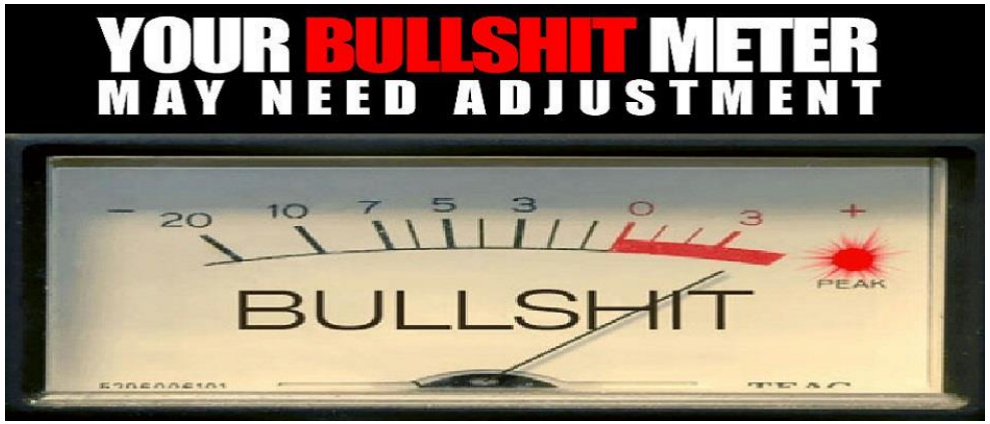
Error handling concepts

- Robust Code / Subsidiarity
- Hierarchical Error Handling / Return Codes
- C++ Exceptions (try / catch)
- Central Error Handler / Error State Machine
- Trap Handling

Robust code

- Your code may not crash, no matter what data it gets or what the state of the system is.

```
void screen::print()
{
          for (uint16_t i = 0; i < m_fillLevel; i++}
          {
                    //m_pGraphicElement should only contain valid pointers
                    m_pGraphicElement[i]->print();
          }
}
```




```
void screen::print()
{
          for (uint16_t i = 0; i < m_fillLevel; i++}
          {
                    //m_pGraphicElement should only contain valid pointers - but we never trust!
                    if (m_pGraphicElement != 0) m_pGraphicElement[i]->print();
          }
}
```

Subsidiarity principle

- Problems should be solved on the lowest possible level.
- If they cannot be solved, the next hierarchical level must be involved.

Example: database search

- A string is passed to a navigation map function in order to find an entry in the database
- The following faults shall be considered
  - The string needs to be trimmed, i.e. no whitespaces at the start and end of the string
  - A database search can only be performed if a database is connected
  - A record can be found or not

Subsidiarity principle
Database search code snippet

```
RC_t searchLocation(string name)
{
        RC_t result = RC_error;

        //Make sure that string is trimmed
        trim(name);

        //Check if database is connected
        if (0 == m_pDatabase){
                CentralErrorHandler("No database connected");
        }
        else{
                int16_t index = m_pDatabaseGetIndex(name);

                //Check if record has been found
                if (-1 == index) {
                        result = RC_recordNotFound;
                }
                else {
                        result = RC_SUCCESS;
                }
        }
        return result;
}
```

Subsidiarity principle
Different escalation levels

The string needs to be trimmed, i.e. no whitespaces at the start and end of the string

- This is something trivial, which can be corrected within the responsibility of the called function
- No need to inform the caller

A database search can only be performed if a database is connected

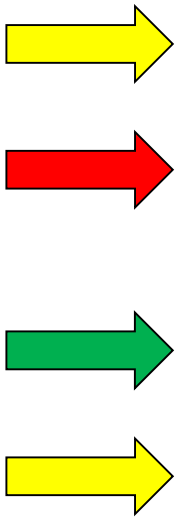
- A not connected database is a pretty heavy problem, something has really gone wrong here
- We do not want to rely on the caller to handle this (although this also would be an option)
- Instead we directly escalate to a central error handler

A record can be found or not

- Typical problem, which should be dealt with by the caller
- Let’s use a return value to inform him.

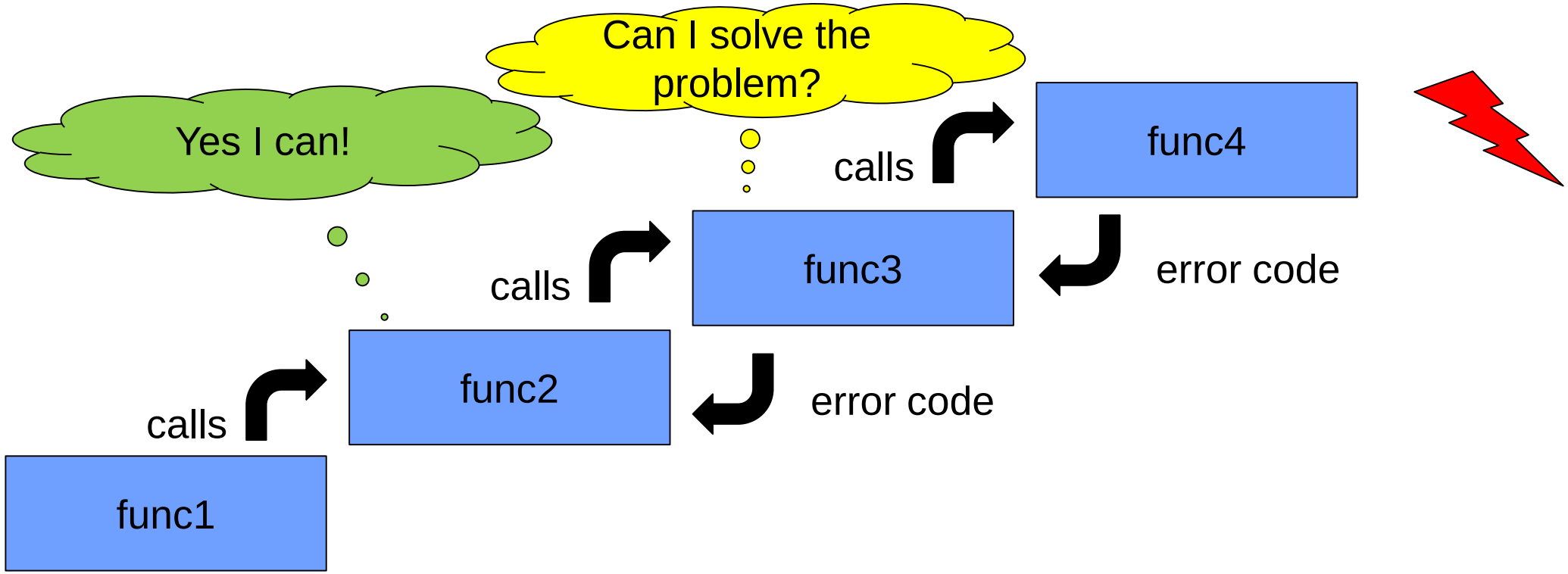
Hierarchical Error Handling
Error Return Code

- The function returns an error code, identifying if the data was processed correctly or not.
- In case of an error indicator, the caller (having a higher hierarchical rank) decides what to do.
- Printing an error message using `cout` or `cerr` is a different story. This may be done in addition, but it never replaces an error code return value!



Type	Advantage	Disadvantage
bool	Very simple, easy to understand	Does not provide too much information
int, string	Provides more values	Rather unspecific, different functions may use it differently, i.e. negative or positive error codes
Central enum	Provides self explaining values	May be inflexible, if the enum does not contain your error
Central enum + local type	Combines precision and flexibility	Requires more complex API

Hierarchical Error Handling
Problem “Chain of Command”



Hierarchical Error Handling

Advantage:

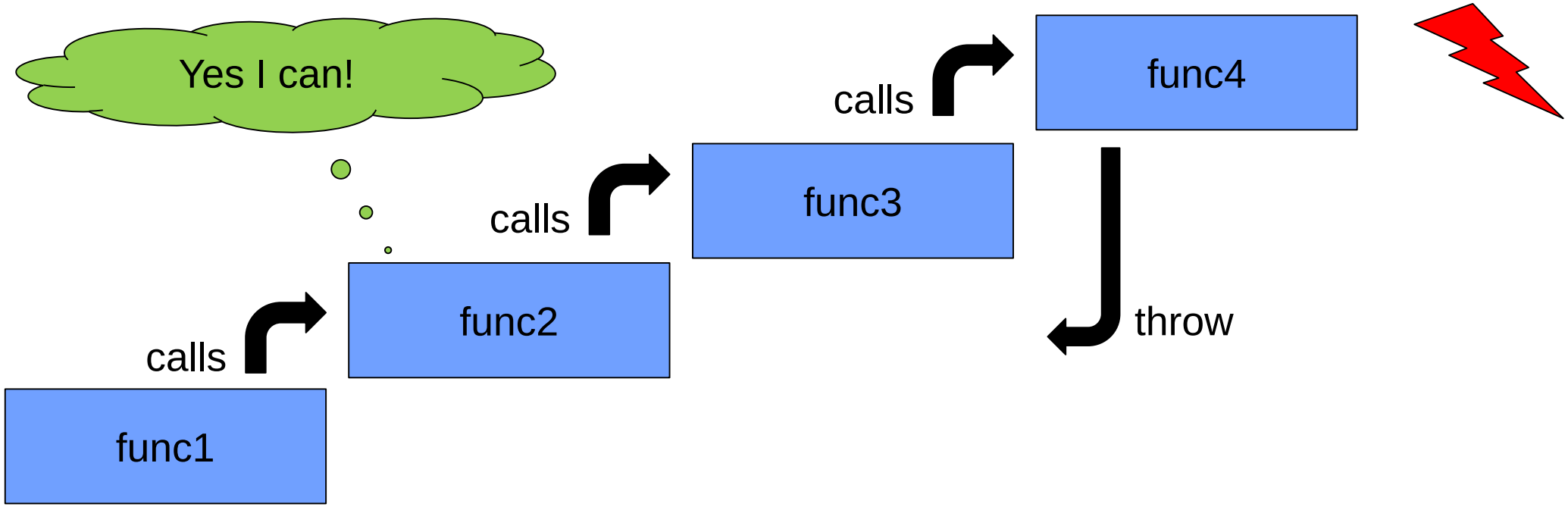
- Problem can be solved on the lowest possible level
- Principle of subsidiarity
- Easy to understand

Disadvantage:

- Complex return value handling, as every call level adds own status information
- Mixture of program logic and exception handling may lead to a reduction of maintainability and performance
- Difficult to use in a big team (unclear responsibilities)

C++ Exception Handling

- The general idea of C++ exception handling is to separate error handling from the normal program flow.
- The exception should be handled without complex return statement hierarchies



C++ Exception Handling

C++ exception handling consists of 4 parts:

- A class (or constant value) representing the exception
- A try clause in which the exception may occur (typically containing the call to lower level functions)
- A catch clause for handling the exception
- And the routine throwing the exception

C++ Exception Handling

```
#define ERROR_A_EQ_ZERO 1
#define  ERROR_B_EQ_ZERO 2

while (1)
{
        cout << "enter a and b:";
        cin >> a >> b;

        try{
                cout << "Result: " << func1(a,b) << endl;
        }
        catch (int &errCode){
                cout << "Error occured: " << errCode << endl;
        }
}

int func1(int a, int b)
{
        if (a==0) throw ERROR_A_EQ_ZERO;
        if (b==0) throw ERROR_B_EQ_ZERO;

        //from here on, a and b have legal values
        return a+b;
}
```

C++ Exception Handling
Own Exception Class

```
Class CException
{
private:
        string m_errorMessage;
        string m_occuredWhere;
        bool m_stopProgram;

public:
        CException(string error, string location, bool halt = true);
        void handle() {
                cerr << m_error << " occurred at << m_location" << endl;
                if (m_stopProgram) terminate();
        };
}

throw CException("Division by zero", "CFilter::calculateAverage()", false);

catch (CException & exception)
{
        exception.handle();
}
```

C++ Exception Handling

Some rules

Implementation

- The catch clause must directly follow the try clause
- Every catch handler only handles one type of exception
- A catch all handler (catch …) must be added to avoid exceptions slipping through
- Exceptions may be escalated to the next level by rethrowing them (throw;)

Design

- Exception handling must be designed right from the start into the system
- All developers in the team must agree on the same approach
- A central exception class is recommended

C++ Exception Handling

Advantages:

- Exception handling code is separated from program logic
- Clear development pattern especially for bigger teams
- Central handling of problems
- Provides error handling for constructors

Disadvantages:

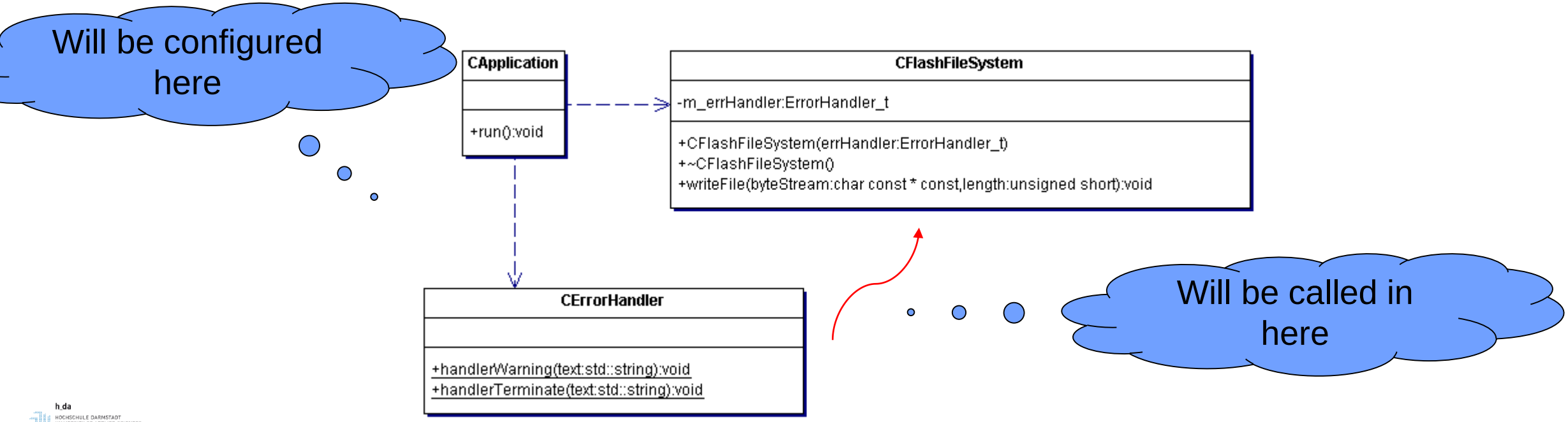
- Risk of stack not unrolled properly
- Requires full C++ language support
- Exceptions may slip through
- Oversized for simple applications

Error Callback

- A function pointer or functor is provided as function parameter or attribute
- In case of an error, this function is being called

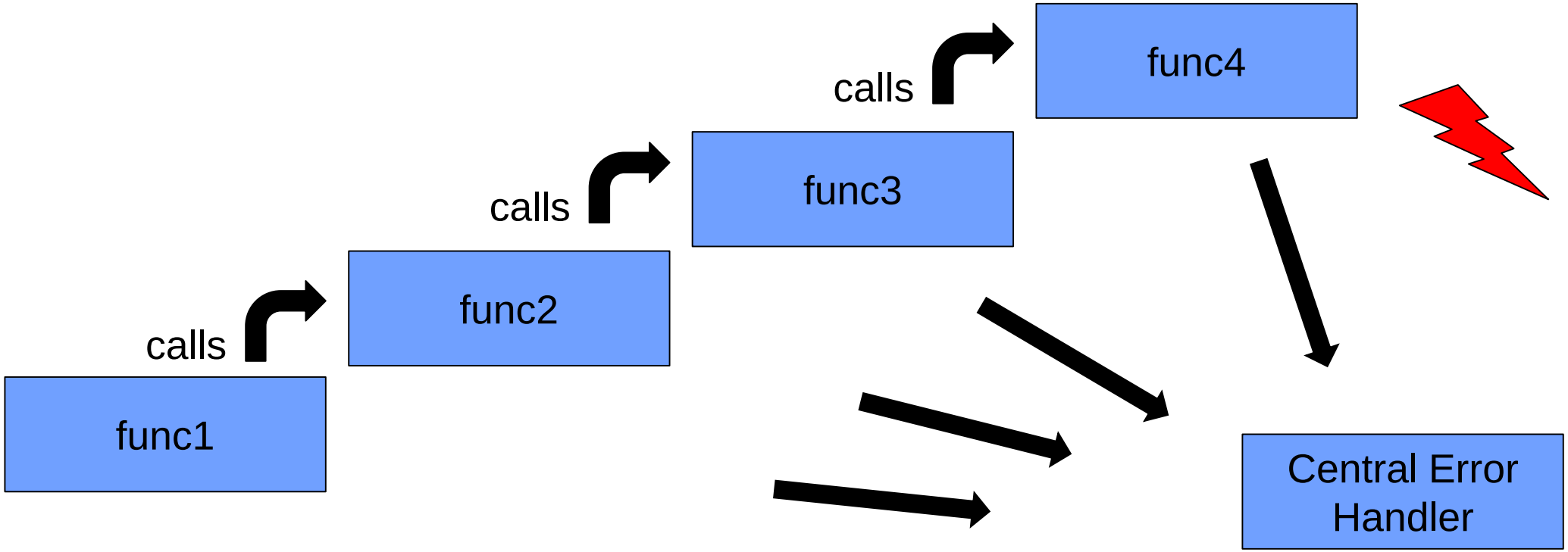
- Pro: Flexibility and error handling inside the function
- Con: A bit more complex

Check previous lecture

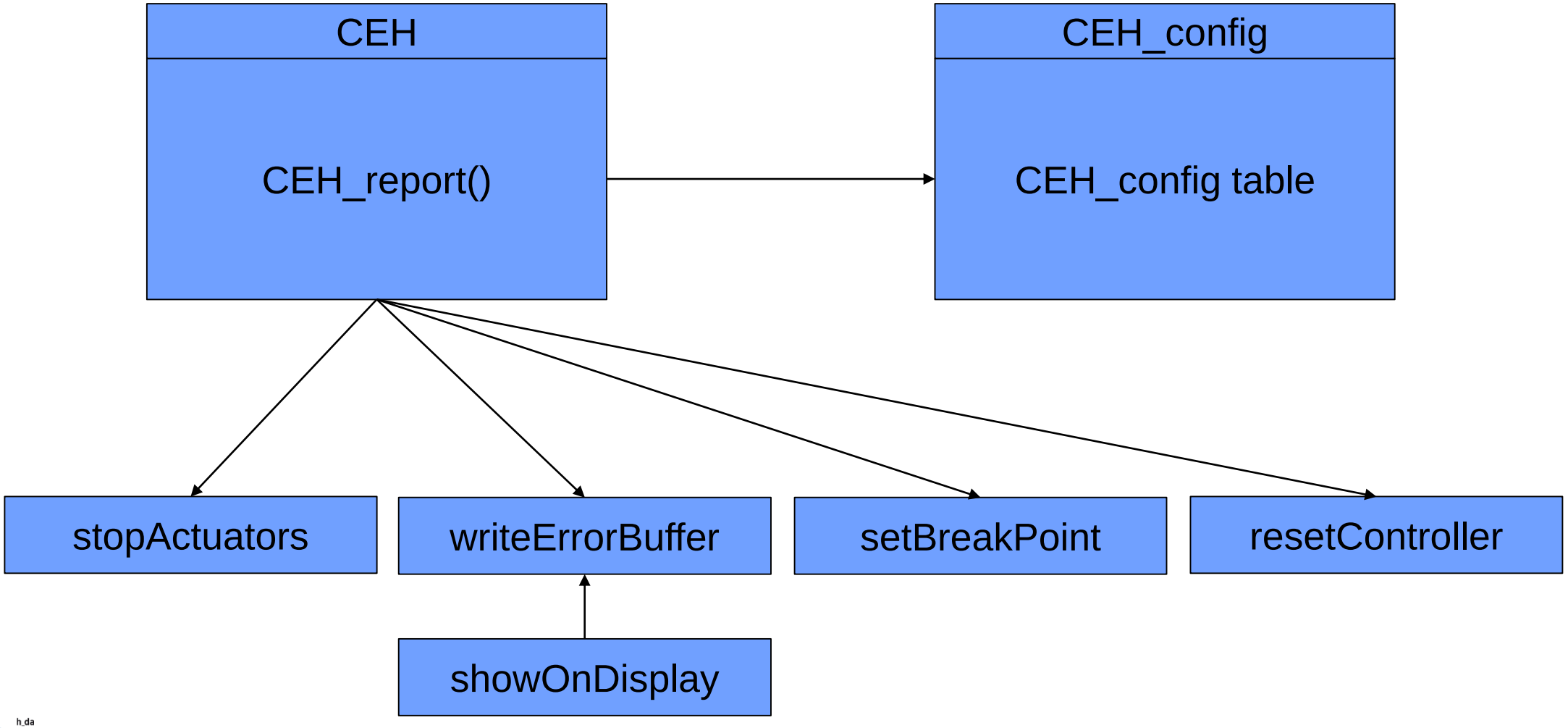


Central Error Handler

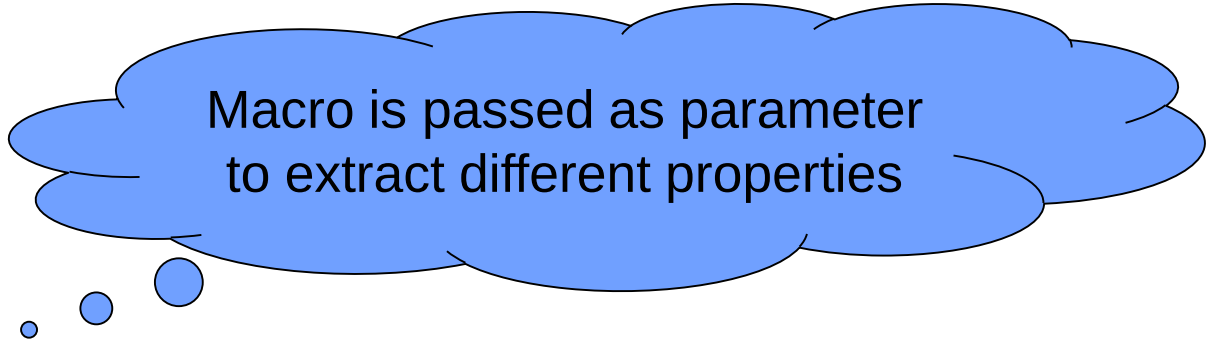
- Errors are reported to a central component, which is responsible for handling them.



Central Error Handler StudentCar

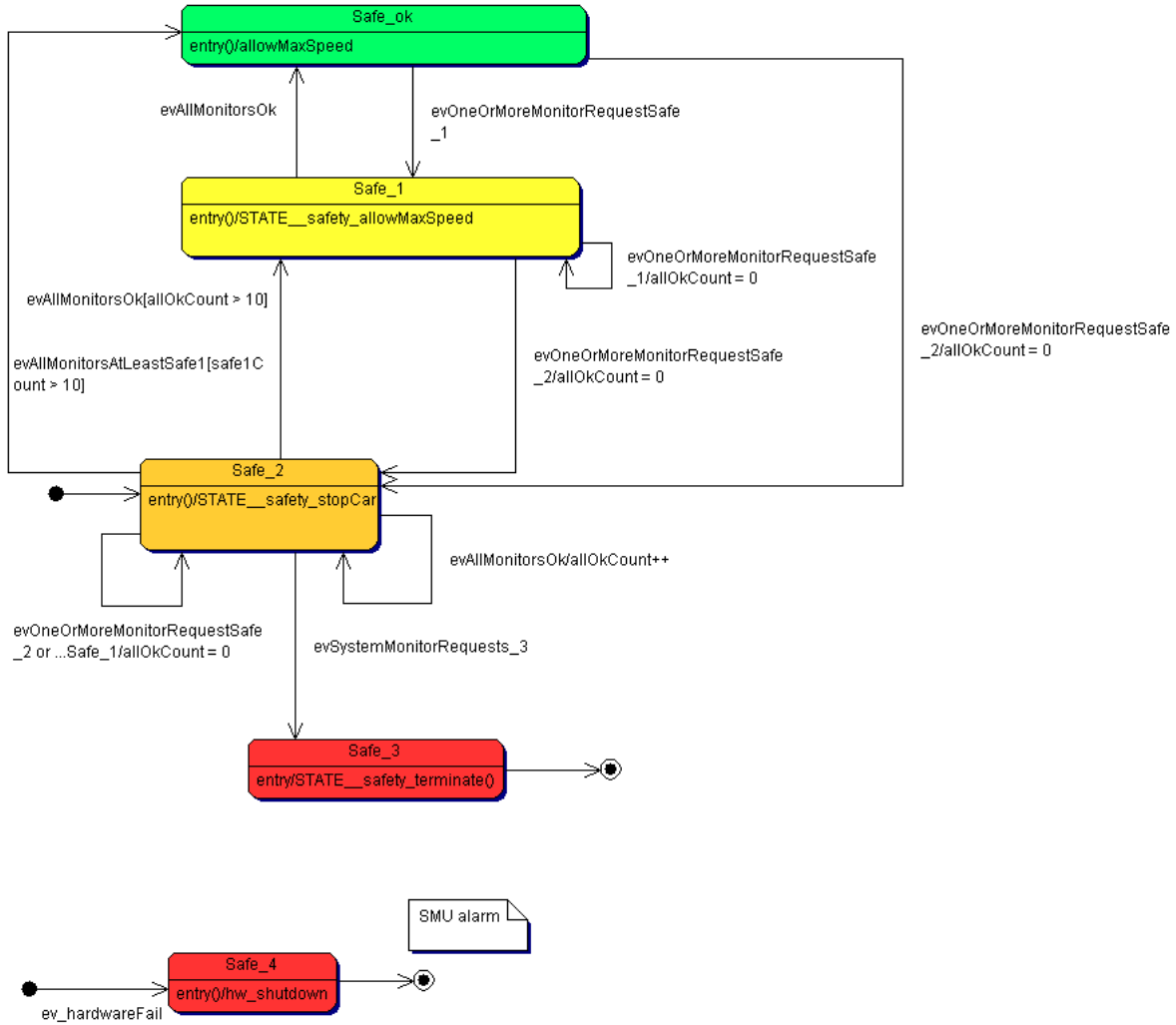


Central Error Handler Configuration




```
//====================   Event Name                      ACTION         STOP_ACTUATORS  SET_DEBUG   ============================//
#define CEH_CFG(DEF)\
        DEF(            EMPTY,                          LOG_BUF,       OFF,            OFF)    /* NOP, do not remove */                                \
        DEF(            HASBOOTED,                      LOG_DISP,      OFF,            OFF)    /* Bootup message */                    \
        DEF(            DRV_INIT,                       STOP,          ON,             OFF)    /* error during driver init sequence*/                  \
        DEF(            SMU_HRALM,                      LOG_BUF,       OFF,            OFF)    /* an SMU Alarm which caused an reset was handled*/     \
        DEF(            SMU_UHRALM,                     STOP,          ON,             ON)     /* an SMU Alarm which caused an reset was not handled*/ \
        DEF(            TRAP_UH,                        STOP,          ON,             ON)     /* Unhandled trap*/                                     \
        DEF(            SETUPTIMER,                     LOG_BUF,       ON,             ON)\
        DEF(            PXBOOT,                         LOG_BUF,       ON,             ON)     /**< \brief Indicates problems with PXROS boot */\
        DEF(            TASK_INTERRUPTVALIDATION,       LOG_DISP,      ON,             ON)     /**< \brief Indicates problems with the task interrupt installation / configuration */\
        DEF(            TASK_CREATENAMESERVER,          LOG_DISP,      ON,             ON)     /**< \brief Indicates problems with the task name server creation */\
        DEF(            TASK_CREATERELEASESERVER,       LOG_DISP,      ON,             ON)     /**< \brief Indicates problems with the task release server creation */\
        DEF(            TASK_CHECKSERVICE,              LOG_DISP,      ON,             ON)     /**< \brief Indicates problems with the task service check e.g. nameserver */\
        DEF(            TASK_REGISTERNAME,              LOG_DISP,      ON,             ON)     /**< \brief Indicates problems with the task id registration with the nameserver */\
        DEF(            TASK_CREATETASK,                LOG_DISP,      ON,             ON)     /**< \brief Indicates problems with the task creation */\
        DEF(            TASK_SIGNALEVENT,               LOG_DISP,      ON,             ON)     /**< \brief Indicates problems with the task activation */\
        DEF(            SYNC_TIMEOUT,                   LOG_DISP,      ON,             ON)     /**< \brief Indicates problems with the task boot synchronisation */\
        DEF(            RTL_WRONGCORE,                  LOG_BUF,       ON,             ON)\
        DEF(            RTL_NOMESSAGE,                  LOG_BUF,       ON,             ON)\
        DEF(            RTL_NOMAILBOX,                  LOG_BUF,       ON,             ON)\
        DEF(            RUN_TIMEOUT1,                   LOG_BUF,       ON,             ON)\
        DEF(            RUN_TIMEOUT2,                   LOG_BUF,       ON,             ON)\
        DEF(            QUERYNAME,                      LOG_BUF,       ON,             ON)\
```

Error State Machine



System is under control, monitors report recoverable errors

Fatal error

Great for escalation of multiple errors.

Trap Handling

- Traps are hardware signals, indicating hardware errors.
- Handling is comparable to interrupt handling
- Post mortem registers can be applied to get additional information on the problem causing the trap
- Typical causes: MPU, alignment, invalid memory, stack

Example Trap Handler

d:\50_HDA\60_Lehre\40_Vorlesungen\25_ADT\20_Lecture\080_ErrorHandling\code\traps\trap_handler.cpp - Notepad++

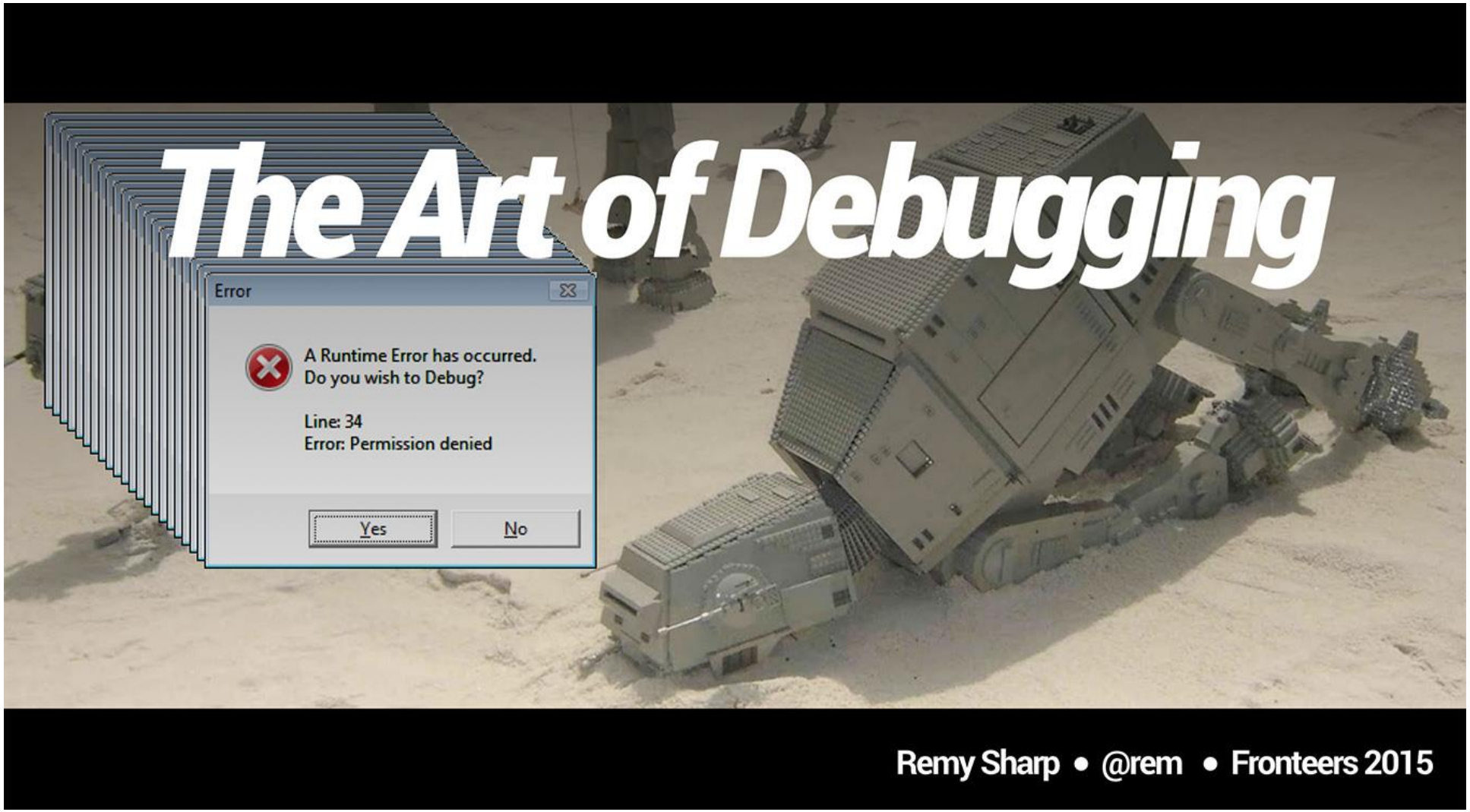
Datei Bearbeiten Suchen Ansicht Kodierung Sprachen Einstellungen Werkzeuge Makro Ausführen Erweiterungen Fenster ?

trap_handler.cpp

```
529         }
530
531     return ret;
532 }
533
534 /* TRAP CLASS - 3   */
535 STATIC TRAP_hdl_ret_t Trap_hdl_3(trap_detail_t* p_details){
536
537     //result of trap handling
538     TRAP_hdl_ret_t ret=TRAP_unhandeled;
539
540     switch (p_details->tin) {
541         case 1:
542             /* Free Context list Depletion Trap (FCD)
543              * The FCD trap is generated after a context save operation, when the operation causes the free context list to become 'almost empty'. The 'a
544              * is signaled when the CSA used for the save operation is the one pointed to by the context list limit register LCX.
545              * The operation responsible for the context save completes normally and then the FCD trap is taken.
546              * See the Architecture Manual vo1 for more information*/
547             LOG_E(TAG, "3.1: Free Context list Depletion Trap (FCD)");
548             TRAP_DETAIL_ACTION
549             break;
550
551         case 2:
552             /* Call Depth Overflow Trap (CDO)
553              * A program attempted to execute a CALL instruction with the Call Depth counter enabled and the call depth count value (PSW.CDC.COUNT) at it
554              * Call Depth Counting guards against context list depletion, by enabling the OS to detect 'runawayrecursion' in executing tasks.*/
555             LOG_E(TAG, "3.2: Call Depth Overflow Trap (CDO)");
556             TRAP_DETAIL_ACTION
557             break;
558
559         case 3:
560             /* Call Depth Underflow Trap occurred (CDU)
561              * A program attempted to execute a RET (return) instruction with the Call Depth counter enabled and the call depth count value (PSW.CDC.COUN
562              * A call depth underflow does not necessarily reflect a software error in the currently executing task. An OS can achieve finer granularity
563              * by using a deliberately narrow Call Depth Counter, and incrementing or decrementing a separate software counter for the current task on* e
564              * or underflow trap. A program error would be indicated only if the software counter were already zero when the CDU trap occurred. */
565             LOG_E(TAG, "3.3: Call Depth Underflow Trap occurred (CDU)");
566             TRAP_DETAIL_ACTION
567             break;
568
```

C++ source file length : 37.736 lines : 810 Ln : 363 Col : 53 Sel : 0 | 0 Windows (CR LF) ANSI INS

Houston, we have a problem!

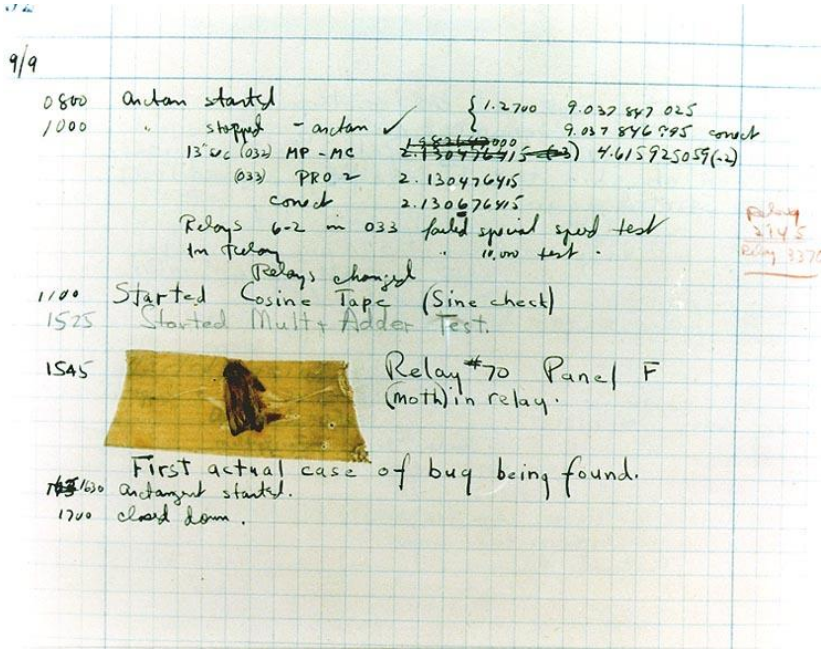


Debugging

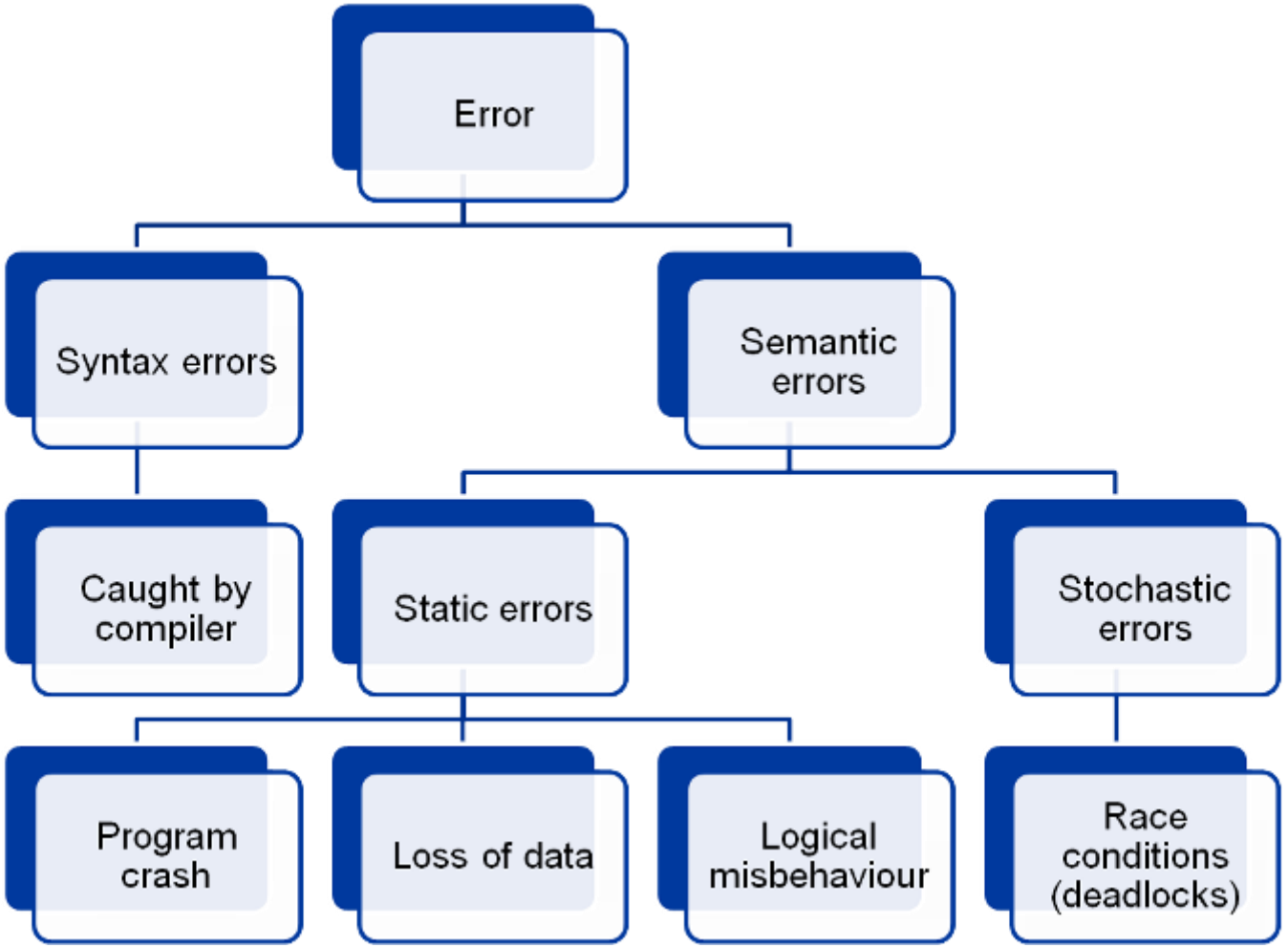
Debugging is the process of finding and resolving defects or problems within a computer program that prevent correct operation of computer software or a system. Debugging tactics can involve interactive debugging, control flow analysis, unit testing, integration testing, log file analysis, monitoring at the application or system level, memory dumps, and profiling.

The terms "bug" and "debugging" are popularly attributed to Admiral Grace Hopper in the 1940s. While she was working on a Mark II computer at Harvard University, her associates discovered a moth stuck in a relay and thereby impeding operation, whereupon she remarked that they were "debugging" the system. However, the term "bug", in the sense of "technical error", dates back at least to 1878 and Thomas Edison.

(Wikipedia)



Error categories

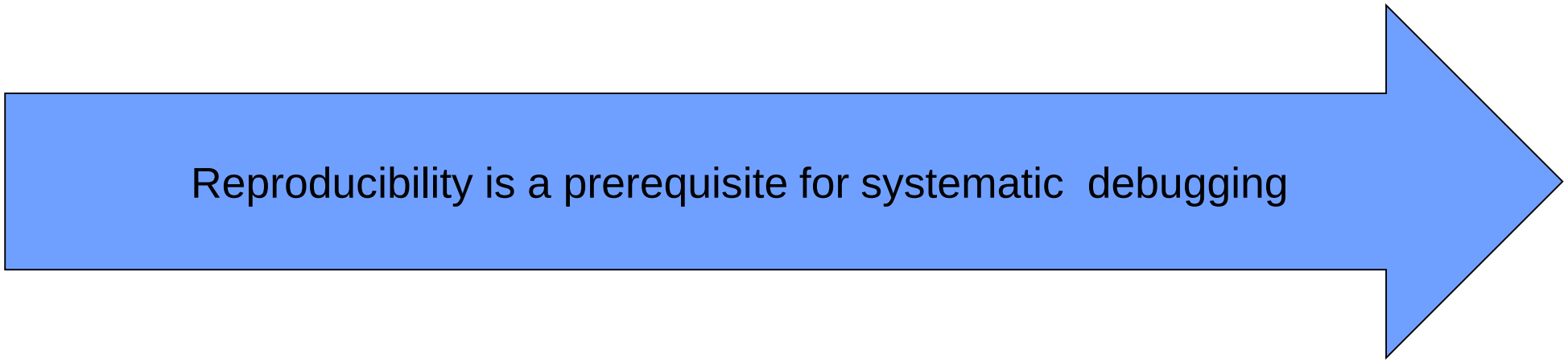


Reproducability

An error is reproducible if the conditions leading to the error are known.

These include

- Program states
- Variable values
- Environmental values (CPU load, memory load,...)
- Timing conditions (Race conditions)



Buggy section

Buggy code is the section of the program which, when fed with the wrong values, leads to a misbehavior of the program.

Example:

```
cin >> a >> b;
result = a/b;
cout << result << endl;
```

b=0???

The buggy section is found by systematic debugging, e. g. using slicing.

Slicing

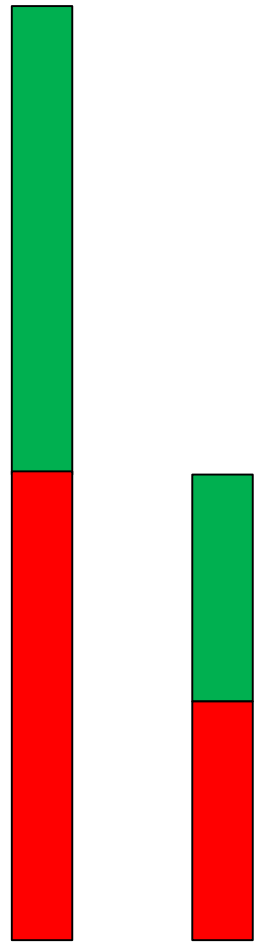
```
void div_errhdl3(float a, float b)
{
    float result;

    errorhdl(b==0,"Parameter b = 0,
             no division possible");

    result = a/b;
    cout << a << " / " << b << " = "
        << result << endl;
}

void div_errhdl4(float a, float b)
{
    float result;

    assert(b==0);

    result = a/b;
    cout << a << " / " << b << " = "
        << result << endl;
}
```

Test ok

Breakpoint

Breakpoint

Test nok

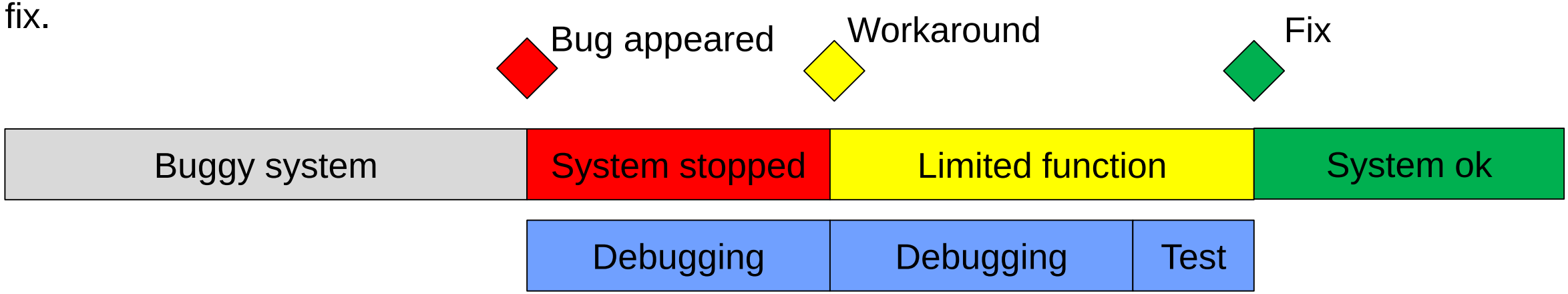


Error Root Cause / Workaround

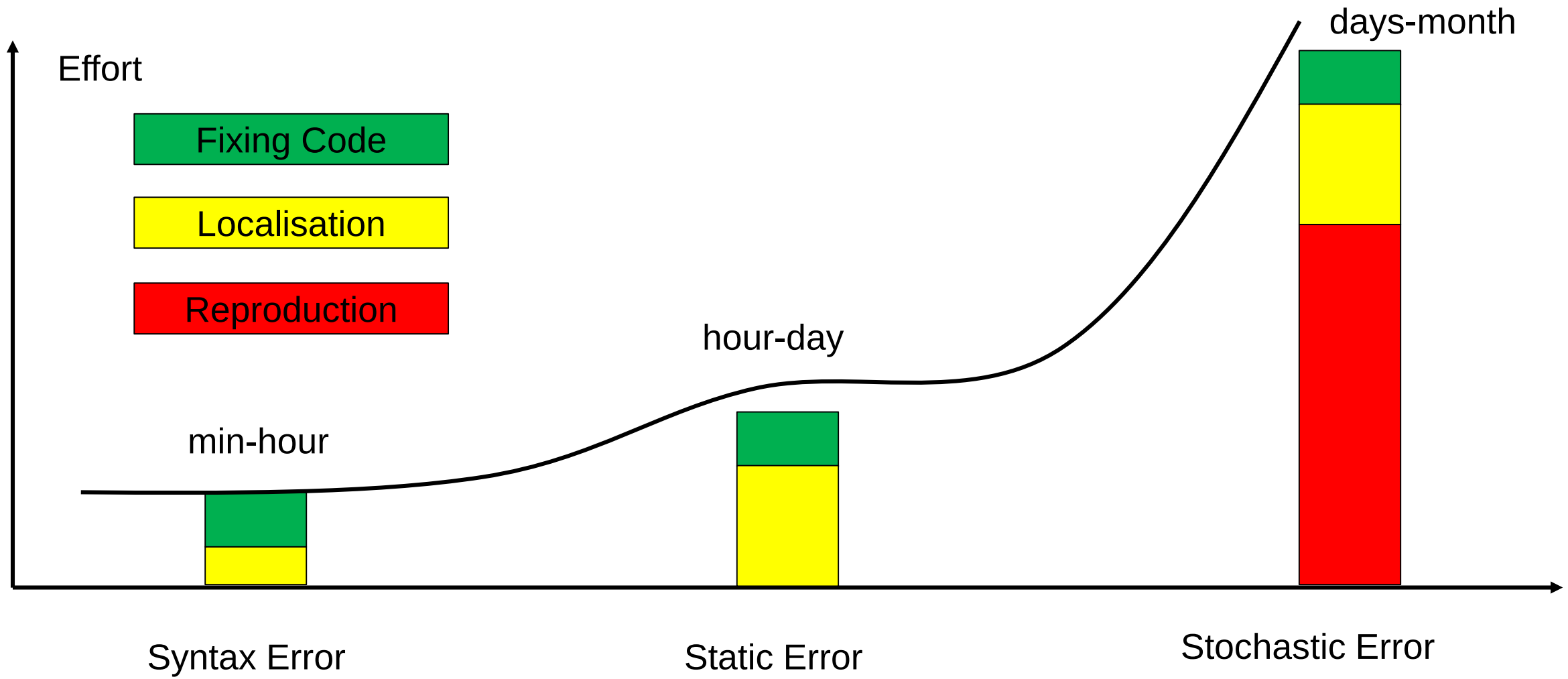
An error root cause is the exact knowledge of the **error conditions** and the **code of the program generating the error** and the **knowledge why this code generated** the error (e. g. because a division by zero is not supported by the compiler)

It is not always possible to find the root cause with reasonable effort. Sometimes we have to accept a **workaround.** A workaround does not solve the root cause, but instead introduces code to minimise the effects of the bug (e.g. by resetting a variable in regular intervals)

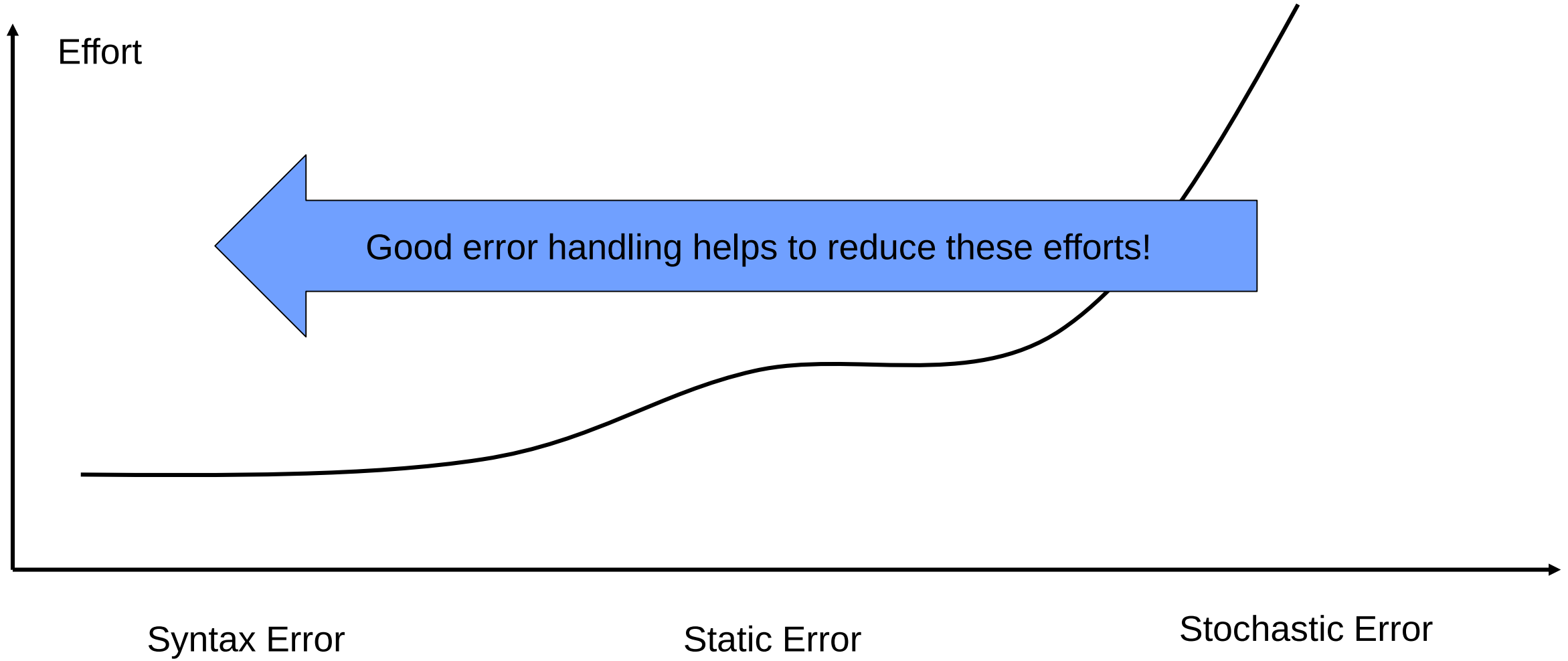
Introducing workarounds do not solve the issue, a certain risk of failure remains. Therefore workarounds should not serve as permanent solutions, but may be used to gain time for a proper fix.



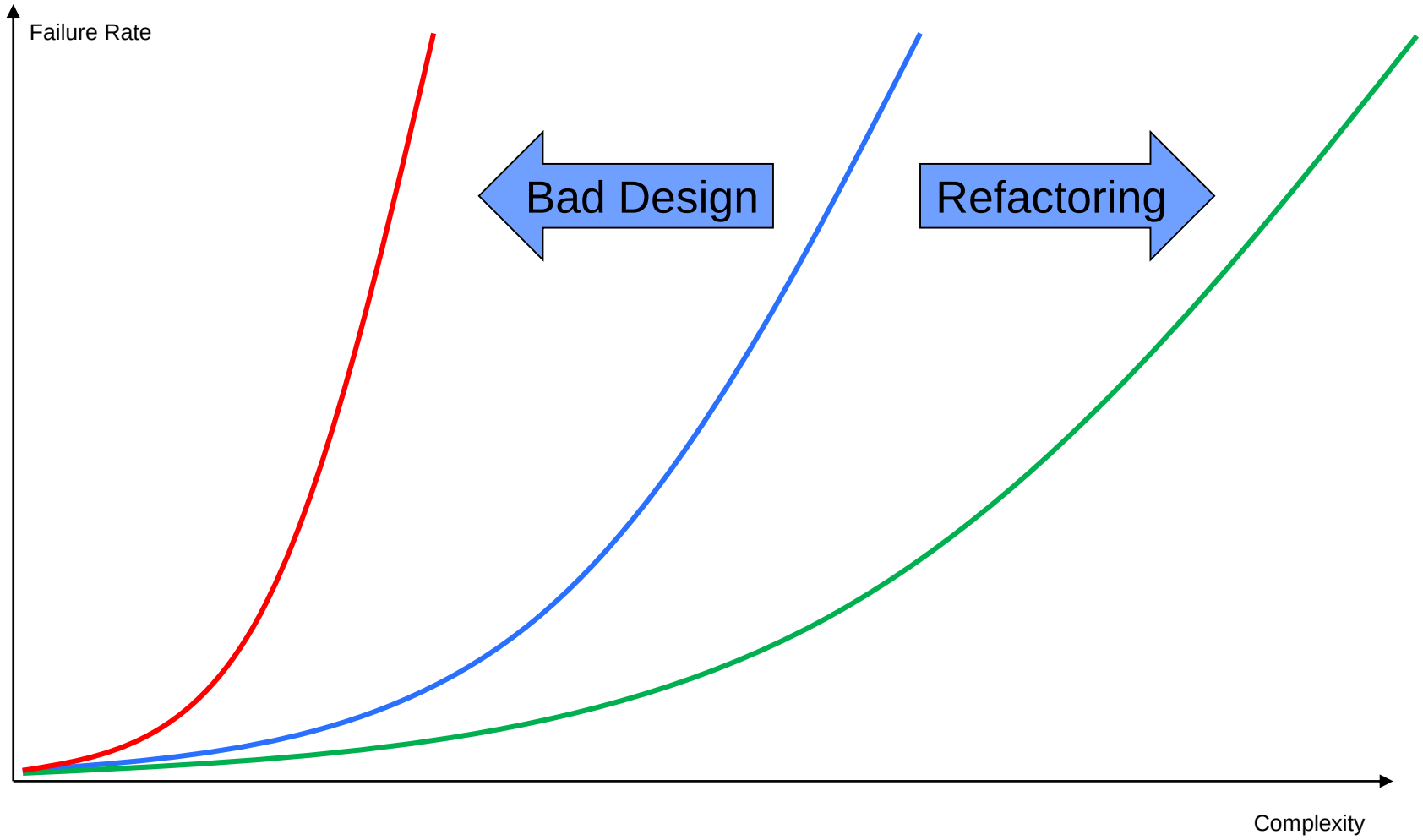
Debugging efforts



Debugging Efforts



Impact of Complexity – we can make our life easier!



Wrap-Up

- Faults, errors and failures
- Analysing sensors, logic, actors and networks / HMI for potential problems
- Identifying faults
- Error handling techniques
  - Hierarchical error handling
  - C++ exceptions
  - Central error handler
- Debugging is (not always) fun